

WebSockets Design Document – Voyago

1. Purpose

Voyago uses WebSockets (via the recommended `socket.io`) to enable real-time, bi-directional updates between multiple users who are viewing or editing the same trip. This avoids frequent polling and allows the server to broadcast updates instantly to clients.

2. How WebSockets will be used in Voyago

2.1 Architecture

- **Backend:** Node + Express + Socket.io (same server/port)
- **Frontend:** React (Vite) + `socket.io-client`
- **Rooms:** one room per trip → `trip:<tripId>`

WebSockets will be used for two main features:

2.2 Collaborative Trip Editing (Real-Time Sync)

When multiple users are on the same trip page:

- If one user edits the itinerary (add/remove/edit activity), flight info, lodging, or trip notes
- The server broadcasts the update
- All other users viewing that trip see changes immediately (no refresh)

2.3 Group Chat (Live Messaging)

For each trip, users can chat in real-time:

- User sends message → server receives → server broadcasts to others
- Messages appear instantly for all participants
- Optional “typing...” and “user joined/left” indicators

2.4 Presence Indicators (Live User Status)

Voyago will display real-time presence indicators to show which users are currently viewing or editing a trip.

When a user joins a trip:

- Their presence is broadcast to other users in the same trip room
- Other users see an updated list of active participants (e.g., online indicator or avatar list)

When a user disconnects or leaves:

- The server broadcasts a presence update
- The user is removed from the active participants list

This enhances collaboration by making trip editing feel live and interactive.

3. High-Level Flow

1. On page load, the client opens a WebSocket connection.
2. The server receives the connection, associates the socket with a trip-specific room, and tracks the user as active in that trip.
3. When a user performs an action (edit itinerary / send chat message), client emits an event.
4. The server receives the event, validates it, persists changes (DB), and broadcasts to other sockets in the same room.
5. Clients receive the broadcast event and update UI state immediately.

4. Socket.io Setup

4.1 Client (One Global Socket Instance)

A single socket instance is created in a separate file so React re-renders do not create duplicate connections.

File: `src/socket.ts`

```
import { io } from "socket.io-client";

export const socket = io(
  import.meta.env.PROD ? undefined : "http://localhost:3000",
  { withCredentials: true } // only needed if using cookies/auth
);
```

In pages/components that need real-time:

- Register listeners in `useEffect(..., [])`
- Cleanup with `socket.off(...)`

5. Rooms Design (Broadcast Only to Sockets on the Same Trip)

We use rooms so broadcasts only reach users viewing the same trip.

5.1 Room Naming

- `trip:<tripId>`

Examples:

- `trip:9f3a...`
- `trip:12345`

5.2 Joining a Room

When the trip page loads, the client emits `trip:join`.

Client

```
socket.emit("trip:join", { tripId });
```

Server

```
socket.on("trip:join", async ({ tripId }) => {  
  socket.join(`trip:${tripId}`);  
});
```

6. Event Design (What Events We Send)

6.1 Collaborative Trip Editing Events

Client → Server

- `itinerary:create`
- `itinerary:update`
- `itinerary:delete`
- `trip:updateFlight`
- `trip:updateLodging`

Payload Pattern

```
{  
  "tripId": "123",  
  "change": { "...": "..." }  
}
```

6.2 Presence Events

Client → Server

- trip:join
- trip:leave

Server → Clients

- presence:update

Presence Update Payload

```
{
  "tripId": "123",
  "activeUsers": [
    { "userId": "u1", "name": "Olivia" },
    { "userId": "u2", "name": "Matt" }
  ]
}
```

When a user joins or disconnects, the server updates the list of active users in the room and broadcasts the updated presence state to all other users in that room.

Server → Clients (Broadcast)

- trip:changed

Broadcast Payload

```
{
  "tripId": "123",
  "type": "itinerary:update",
  "data": { "...": "..." },
  "updatedBy": "userId",
  "updatedAt": "ISO timestamp"
}
```

Broadcast Rule

Broadcast only to others in the same trip room:

```
socket.to('trip:${tripId}').emit("trip:changed", payload);
```

The server should generally persist changes to the database first and then broadcast, so updates represent the source of truth.

6.3 Chat Events

Client → Server

- chat:send

Payload

```
{
  "tripId": "123",
  "message": "hey everyone",
  "clientId": "uuid"
}
```

Server → Clients

- `chat:newMessage`

Payload

```
{
  "tripId": "123",
  "messageId": "db id",
  "userId": "sender",
  "message": "hey everyone",
  "sentAt": "ISO timestamp"
}
```

Optional

- `chat:typing` (for “user is typing...”)
- `presence:join` / `presence:leave` (online indicators)

7. Remembering Users (Auth vs Socket IDs)

Socket IDs are ephemeral and change on refresh. Voyago will identify users via:

- **Logged-in users:** use JWT/session token to identify the user on connection
- **Guest users (optional):** issue a persistent cookie token via an HTTP route and use `withCredentials: true`

On connection, the server can associate:

- `socket.id` → `userId`
- `socket.id` → `activeTripRooms`

This enables permissions checks before accepting edits/messages.

8. Server Responsibilities

For each received event, server must:

1. Validate payload (tripId exists, user belongs to trip, etc.)
2. Apply change in database (or reject)

3. Broadcast resulting update to room
4. Handle disconnect cleanly

9. React Integration Rules

- Create socket once (`socket.ts`)
- Register listeners inside `useEffect` with cleanup
- Update local state in response to events (or refetch trip snapshot if needed)

Two valid UI update strategies:

1. **Patch updates:** apply received change directly to local state
2. **Snapshot refresh:** on `trip:changed`, refetch trip from REST API for consistency (simpler, slightly more network)

10. Deployment Notes

When deployed behind a reverse proxy (Caddy), WebSocket upgrade must be forwarded to Node. The Caddyfile must support WebSocket proxying for the Node server route.

11. Summary

WebSockets in Voyago enable:

- Real-time collaborative trip planning (everyone stays in sync)
- Real-time group chat inside each trip
- Room-based broadcasting so updates only go to users on the same trip page
- User identification via token/cookies rather than socket ID